

June 15th, 2024

Timer SysTick Assignment Report

Embedded System 3, LAB 03

Document Version 1.0

Edited by:

Farros Ramzy (3767353)

Abstract

This technical report describes about the basic use of bit registers in the STM32 board to read an input and write an output, and about the clock timer register of SysTick in ARM4 microcontroller that built in the Nucleo board together with the STM32. This report focused on the procedure, context, and knowledge about the SysTick registers of the ARM4 in the STM32-F303RE board.

List of Figures

Figure 2. 1. 1 STM32-F303RE Board pinout.	2
Figure 2. 1. 2 A wiring diagram for the SysTick Experiment.	3
Figure 2. 1. 3 System Clock Configurations.....	3
Figure 2. 2. 1 The SysTick Clock CONTROL register.	4
Figure 2. 2. 2 The LOAD register of the SysTick.	4
Figure 2. 2. 3 The VALUE register of the SysTick.	4
Figure 2. 4. 1 A wiring diagram of the SysTick experiment.	7
Figure 3. 1 single sweep logic analyzer test.....	8
Figure 3. 2 multiple sweep logic analyzer test.....	8

List of Tables

Table 2. 2. 1 SysTick CONTROL pin details.	4
---	---

Table of Contents

1.	Introduction	1
1.1.	Equipment.....	1
2.	Procedure.....	2
2.1.	Preparation	2
2.2.	Part A, SysTick Clock Source	4
2.3.	Part B, SysTick Interrupt Handler.....	6
2.4.	Part C, Experiment	7
3.	Experiment Result.....	8
4.	Conclusion.....	9
5.	Recommendation.....	9
6.	References	10

1. Introduction

This practicum is a learning method to understand the use of the system timer (SysTick) register on the ARM4 microcontroller that attached to the STM32 board.

The goal of this project is to understand the basic knowledge of the SysTick and the interrupt handling of the Nucleo board. And to do this, this project is divided into three parts of assignment, which are the testing practice of the basic SysTick clock source register, the testing practice of the SysTick interrupt handler, and an experiment to use that SysTick interrupt handler in a small project.

And to do these, practice research by using the LED must be done.

The document will explain each part in the Procedure section, the result will be placed in the Conclusion section, and at least, a recommendation will be made in the Recommendation section. Some references that have been used for the practicum and included in this document are listed in the References part by the end of this report.

1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. An STM32-F303RE board
3. At least one LEDs (can add more for the experiment purpose)
4. Two pushbuttons (only for experiment purpose)
5. A breadboard
6. A bunch of wires
7. One $470\ \Omega$, $\pm 5\%$ resistor for each one of the LEDs
8. One $1000\ \Omega$, $\pm 5\%$ resistors for each one of the buttons (if applied)

2. Procedure

2.1. Preparation

To start the assignment, it is required to read the GPIO registers in the reference manual of the STM32-F303RE board which is listed in this References section.

After that, to understand the pin position that will be used from this STM32 board, the Nucleo User Manual document must be read.

From that manual, it can be known that the STM32-F303RE pins that can be used are presented in this Figure 2. 1. 1 below.

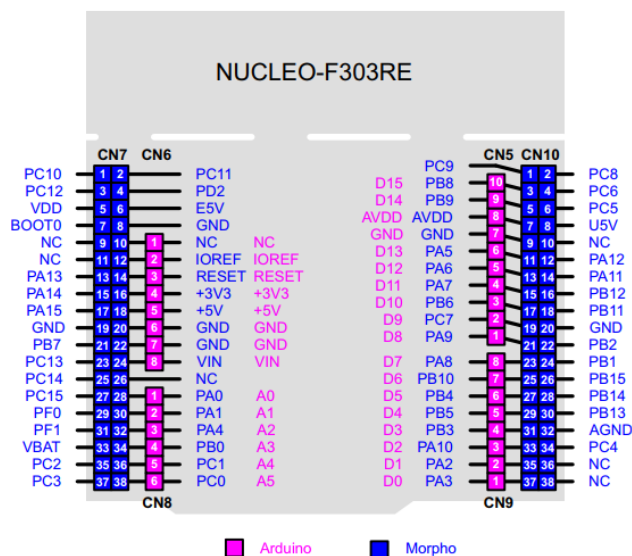


Figure 2. 1. 1 STM32-F303RE Board pinout.

From this Figure 2. 1. 1, it is visible that the pink or purple pins from the board is usable for the Arduino programming.

However, in this practicum, the pins that can be used for the STM GPIO register manipulation are the Morpho pins, reserved for the Nucleo STM32 board.

Nevertheless, there are some pins that cannot be used for external interference such as the internal LED pin, PA5 or PB13, and the internal button, PC13 because these pins are internally built in for the board.

After decided the output pin, the tools such as an LED, a resistor, and some jumper wires must be prepared.

After that, it is recommended that to read the Cortex M4 generic guide document to understand where the ARM4 register is located, and how to use them.

If everything is prepared, the LED can be placed on the Nucleo board such is shown in this Figure 2. 1. 2 below.

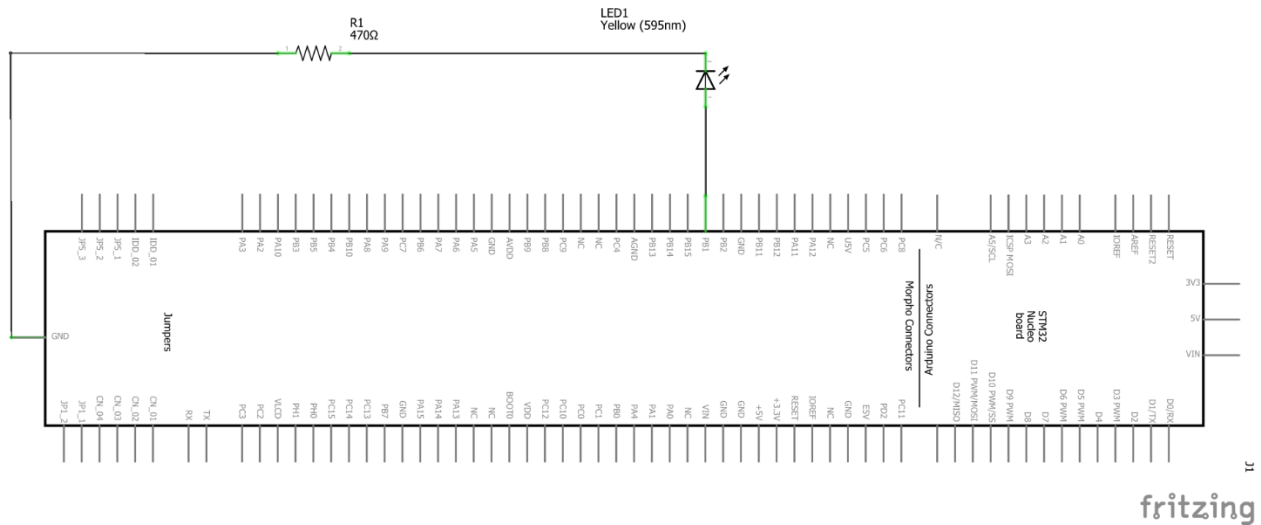


Figure 2. 1. 2 A wiring diagram for the SysTick Experiment.

To be informed, based on this Figure 2. 1. 2 above, the practicum only needs an LED that is placed on the PB1 of the board. No more output pins are needed because the point of this practicum is just to understand the use of the system timer (SysTick) register.

And this goal can be reached by blinking the diode on PB1 using the SysTick timer instead of using the HAL_Delay function which is already provided by the STM32 HAL_GPIO library.

After preparing the board, now is the time to setup the clock configuration of the system.

This clock configuration can be done in the ioc file of the STM project IDE, shown in this Figure 2. 1. 3 below.

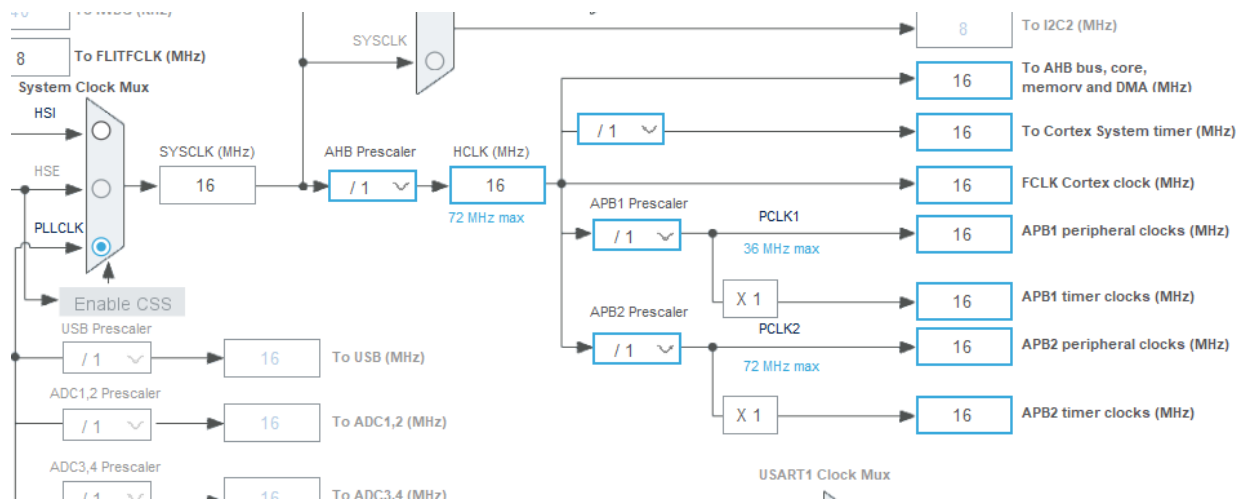


Figure 2. 1. 3 System Clock Configurations.

In this Figure 2. 1. 3 above the system clock is setup to 16 MHz, following to the assignment requirement, and the AHB clock set with no division, since the system will only use the peripheral bus with the ARM Cortex timer to controls the input and output pins.

2.2. Part A, SysTick Clock Source

After ready with the preparation, to test the SysTick's registers, some setups must be made.

The first thing needs to be setup is the output register (the yellow LED in this case, based on the Figure 2. 1. 2).

This setup can be done by setting up the GPIO pin, MODE register, and TYPE register as an output in the setup of the main program.

After that, the next thing to do is to setup the SysTick registers. However, some important things must be understood first before doing the setup of this SysTick register.

The SysTick registers use at least three lines of registers to operate the time control system of the microcontroller. The first one is the CONTROL register which can be seen in this Figure 2. 2. 1 below, the second one is the RELOADER register which is presented in this Figure 2. 2. 2 below, and the last one is the VAL register which is shown in this Figure 2. 2. 3 below.

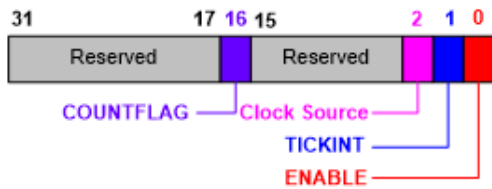


Figure 2. 2. 1 The SysTick Clock CONTROL register.

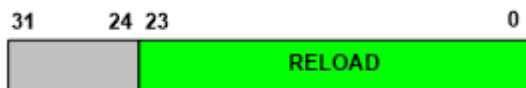


Figure 2. 2. 2 The LOAD register of the SysTick.



Figure 2. 2. 3 The VALUE register of the SysTick.

From the Figure 2. 2. 1 above, it is understandable that the CONTROL register is using four pins to set the SysTick status. The pins can be described in the Table 2. 2. 1 as follows:

Table 2. 2. 1 SysTick CONTROL pin details.

Pin	Use	Description
0	Enable	This pin is used to enable the CONTROL register of the SysTick.
1	TickInt (Interrupt)	This pin is used to set the SysTick to use an exception request, which is included as an interrupt, and can be handled by the interrupt function which called as the interrupt handler.

2	Clock Source	This pin is indicating the clock source (external or processor clock).
16	Flag	This pin is a <u>read only</u> pin, used to set the timer change only if the flag is being read by the clock configurator of the SysTick.

And because this part of the practicum did not use the interrupt, the pin value that can be used to set the CONTROL register of the SysTick is 0b101.

And after setting the CONTROL register, the interrupt priority of the SysTick register must be set to the least urgency by using the NVIC_SetPriority() function, and make its priority value to the largest.

After setting up the CONTROL register, the LOAD register can be set to calculate the count down per millisecond. Because the system is using the 16 MHz timer frequency, the LOAD register is supposed to be 16000 – 1 which make the program can use the bit register as much as possible to the 16 MHz frequency. The “– 1” is needed to be added to the setup because the system is using the 16000 bits register from 0b0000.

Once it is done, the SysTick can be used in the customized delay function such as DelayMS() to be used for the entire program.

The value of the DelayMS can be vary, and it is visible by using it to blink the LED output on the PB1 pin of GPIOB register of the STM32 board.

2.3. Part B, SysTick Interrupt Handler

This part is as the same as the Part A, SysTick Clock Source above. However, instead of only using the Clock Source of the CONTROL register of the SysTick, in this case, the TickInt pin for the interrupt handler is also used.

To do this, the SysTick register must be written as 0b111 instead of 0b101.

And after setting up the SysTick and the GPIO, the program must use a handler function to operates the interrupt.

In the code, this function is a void function, named as the SysTick_Handler().

Inside the function, a method to count down the interrupt can be placed, which will made the program to do the countdown instead of only using the Clock Source to do it.

2.4. Part C, Experiment

This experiment in this practicum is literally using the previous experiment system in the first lab assignment. The diagram to build this system for the SysTick experiment is not changed from what was done in the LED Push-button assignment's experiment, like what is presented in this Figure 2. 4. 1 below.

However, what changed is the setup of the code.

In this practicum, the LED setup to set and reset the output, as the same as the push-button setup as the input are placed outside of the main function, using each of their own functions.

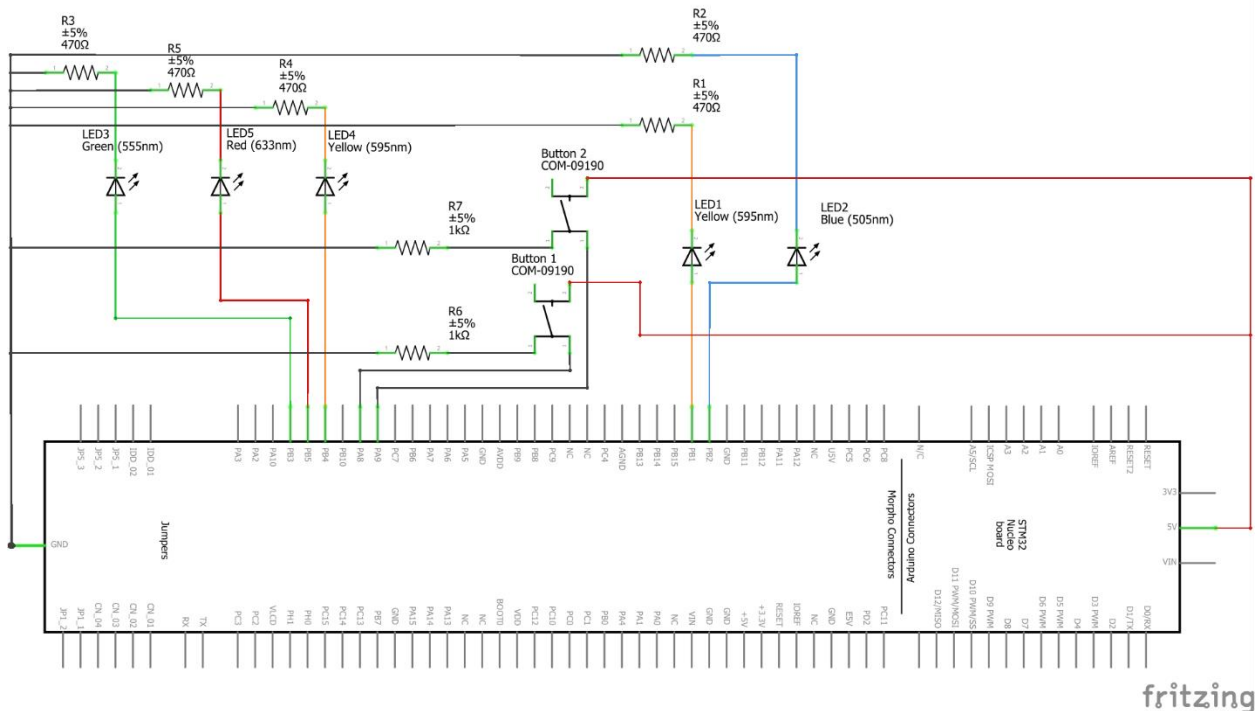


Figure 2. 4. 1 A wiring diagram of the SysTick experiment.

The button setup is placed in the SysTick_Button(), and the LED setup is placed in the SysTick_LED().

And other than the creation of these functions, the goal of this experiment is to proof if the SysTick register can be used as the same way as the HAL_Delay() without breaking the program. Therefore, the SysTick setup function is also created in the program.

And as an addition, the CONTROL register is set up to the 0b111 to make use of the interrupt handler, rather than only using the Clock Source program of the SysTick controller.

3. Experiment Result

After doing the experiment, a result based on the logic analyzer test can be seen in these figures below. The timer value that was used to test on the logic analyzer is 25 ms for each LEDs.

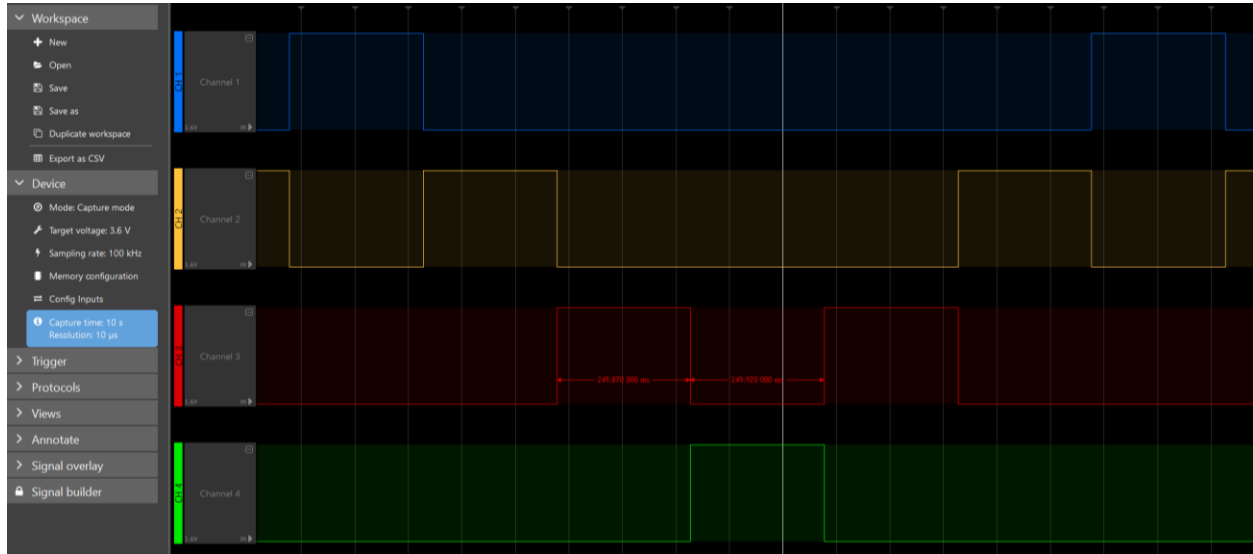


Figure 3. 1 single sweep logic analyzer test.

This Figure 3. 1 above is the test result on the experiment (multiple LEDs). It is shown that each LED is doing the single sweep where they are being turned on and off every 250 ms after the previous LEDs turn.

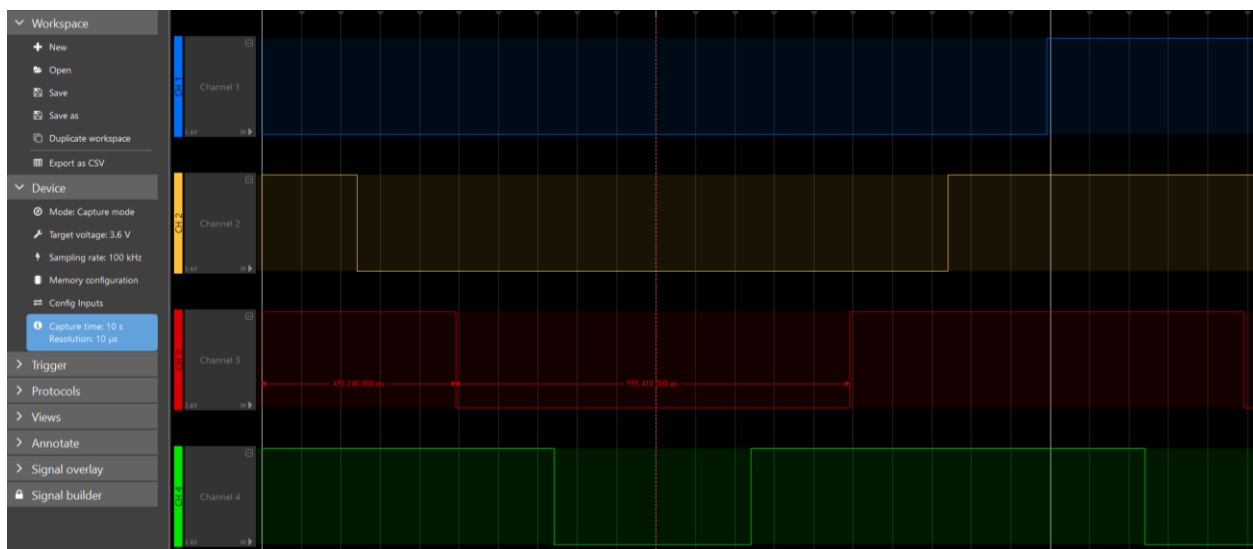


Figure 3. 2 multiple sweep logic analyzer test.

And in this Figure 3. 2 above, the LED rows are turned on and off in a multiple LEDs' sweep. The LEDs are turned off every 250 ms after the previous turn, then they are switched on every 250 ms after the previous turn.

4. Conclusion

After doing this practicum, it is understandable that the SysTick clock configuration is depending on the clock frequency. It is because the LOAD register limits the SysTick count down range based on the maximum clock configuration of the microcontroller in its setup.

If the SysTick LOAD value is higher than the clock configuration, the system will not be able to detect where it should reload the timer.

And after doing the experiment, it is proved that the SysTick configuration could replace the HAL_Delay() function from the HAL_GPIO library without defecting the program.

The system timer is worked as efficient as the HAL_Delay(). However, the LOAD register setup of the timer must be done manually, unlike the "HAL" functions which are already preconfigured when the system is being built.

5. Recommendation

The SysTick function can be useful if the interrupt handler is extended, to the point where it can be used as a read input interrupt instead of just the timer tick interrupt so the input like the two buttons which what was done in the experiment did not block the output workflow, like the LED's blinks.

6. References

Along this practicum, such documents and references are being used.

- Es3_lab_02 report, March 2024
- Cortex- M4 Devices Generic User Guide (ARM), 2010
- DM00043574-ReferenceManual_STM32F303RE(www.st.com), January 2017
- UM1724 USER manual STM32 Nucleo-64 boards (MB1136) (www.st.com), April 2019
- Arduino Pull-up Pull-down Resistor (https://arduinogetstarted.com/fag/arduino-pull-up-pull-down-resistor?utm_content=cmp-true)
- Resistor Color Code Calculator (<https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code>)

Git link to access the source file of the assignment:

- SYSTICK:

<https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/3-timers>